CLASSES

AND OBJECTS

CLASSES

You must be able to

1. Understand the difference between classes and objects
2. Create a simple class
3. Understand and use private and public access specifiers
4. Code attributes and methods
5. Code, and use, constructors (No arguments and with arguments)
6. Getter and setter methods
7. Code the toString () – overriding the method defined in the Object class
8. Code a simple class from an UML class diagram
9. Code worker methods for a class.

CLASSES

You must be able to

Code a main() where

1. an object or objects of a class is created using the different constructors
2. getters()/setters() are used
3. toString () is called to display the details of the class
4. worker methods are called
5. add objects to an array and use the array of objects

CLASSES

The **principles of OOP** are:

1. Encapsulation
2. Abstraction
3. Inheritance and
4. Polymorphism

CLASSES

What are these packages, classes and methods?

Load your Java Docs and view as frames

http://localhost:8080/javadocs/

CLASSES

Example 1: main method

Java main() method is always **static**, so that compiler can call it without the creation of an **object** or before the creation of an **object** of the **class**. In any **Java** program, the **main() method** is the starting point from where compiler starts program execution. So, the compiler needs to call the **main() method**

CLASSES

ACCESS MODIFIERS

Private

Protected

Public

Private Access Modifier - Private

Methods, variables, and constructors that are declared private can only be accessed within the declared class itself.

Private access modifier is the most restrictive access level. Class and interfaces cannot be private.

Variables that are declared private can be accessed outside the class, if public getter methods are present in the class.

Using the private modifier is the main way that an object encapsulates itself and hides data from the outside world

```
private int age=0;
```

Protected Access Modifier – Protected

Variables, methods, and constructors, which are declared protected in a superclass can be accessed by any class within the package of the protected members' class.

The protected access modifier cannot be applied to class and interfaces. Methods, fields can be declared protected, however methods and fields in a interface cannot be declared protected.

Protected access gives the subclass a chance to use the helper method or variable, while preventing a nonrelated class from trying to use it.

```
protected Boolean processOrder() { return true;}
```

Public Access Modifier - Public

A class, method, constructor, interface, etc. declared public can be accessed from any other class. Therefore, fields, methods, blocks declared inside a public class can be accessed from any class belonging to the Java Universe.

However, if the public class we are trying to access is in a different package, then the public class still needs to be imported. Because of class inheritance, all public methods and variables of a class are inherited by its subclasses.

```
public static void mina(String [] args)
```

Default Access Modifier - No Keyword

Default access modifier means we do not explicitly declare an access modifier for a class, field, method, etc.

A variable or method declared without any access control modifier is available to any other class in the same package. The fields in an interface are implicitly public static final and the methods in an interface are by default public

```
boolean processOrder() { return true;}
```

CLASSES and OBJECT

class	objects
Fruit	Apple
	Banana
	Mango

Another example:

class	objects
Car	Volvo
	Audi
	Toyota

Reference: https://www.w3schools.com/java/java_oop.asp

CLASS and OBJECT

CLASSES

- Let's create a Java Project: myFirstClassProject
- Let's print "Hello World" in the main method AND run the project
- Create a class called aPerson – as part of the myFirstClassProject package
- Create a constructor aPerson as a special method – check code
- Complete the UML diagram
- What are setters and getters?
- Homework on next slide!

CLASSES

HOMEWORK

Program 4 (Under Classes – Student Account)

Create the Methods and test the Java application

Methods are:

pay(double): double

addFees(double): double

refund(): void

Test your StudentAccount class by creating objects and calling the methods addFees(), pay() and refund(). Use toString() to display the object's data after each method called